

Pullrun: A Content-Addressed Workload Execution System

Unifying Containers, MicroVMs, and Apple Silicon VMs via Zero-Copy DAG Storage

Mohammed Boukaba
mo.boukaba@gmail.com

Version 0.3.2 — June 2026

Abstract

We present PULLRUN, a content-addressed workload execution system that treats the OCI (Open Container Initiative) image format as a universal workload substrate. Unlike existing container runtimes that require separate build pipelines for containers and virtual machines, PULLRUN boots the same OCI image as a Linux container (via **runc**), a Firecracker microVM, or an Apple Silicon VM—with no image conversion or repackaging.

pullrun version 0.3.2 is built on seven core contributions: (i) a **zero-copy DAG store** that replaces tar extraction and overlaysfs with memory-mapped, content-addressed node graphs using rkyv serialization, achieving first-pull latencies of **968 ms** (2× faster than Docker) and idle daemon memory of **24.6 MiB**; (ii) a **unified executor abstraction** that decouples workload specification from backend selection, allowing the same OCI manifest to materialize as a **runc** container, a KVM-based Firecracker microVM, or a macOS Virtualization.framework VM; (iii) a **P2P block distribution layer** that enables cluster-wide image dissemination via Bloom-filtered delta synchronization over gRPC, eliminating the registry bottleneck for multi-node deployments; (iv) a **policy engine** providing defense-in-depth image gating via cosign signature verification, CycloneDX SBOM analysis, and CVSS-based vulnerability scoring; (v) a **native MCP server** that exposes every runtime operation as a Model Context Protocol tool, enabling AI agents (Claude Code, Cursor, opencode) to pull images, manage workloads, and inspect the runtime through natural language; (vi) a **Windows/WSL2 backend** that extends full container and Firecracker microVM support to Windows 10/11 via the Windows Subsystem for Linux 2, sharing the identical DAG store and CLI across all three host platforms; and (vii) a **unified network model** that places all workloads—containers and VMs alike—on a shared L2 segment with an in-process userspace proxy, DNS server, and IPAM. We evaluate PULLRUN across all backends on Linux (x86-64, ARM64), macOS (Apple Silicon), and Windows (WSL2 x86-64), demonstrating VM boot times of **160 ms** (Apple Virtualization), container run latency of **400 ms**, P2P sync throughput 4.6× faster than registry pulls, and a daemon memory footprint of **24.6 MiB** RSS at idle. PULLRUN 0.3.2 is open-source at <https://github.com/pullrun/pullrun>. A citable archive of the source corresponding to this report is available at <https://doi.org/10.5281/zenodo.20679669> (DOI: 10.5281/zenodo.20679669).

1 Introduction

The OCI image format has become the *lingua franca* of modern workload distribution. Tens of billions of container image pulls occur annually [1], driven by an ecosystem spanning development, CI/CD, and production deployment. However, the current container stack suffers from a fundamental architectural tension: the image format, runtime, and isolation mechanism are tightly coupled. This coupling manifests in three critical ways that constrain how operators deploy and manage workloads.

The extraction tax. Docker and containerd mandate overlaysfs or equivalent union filesystems for local storage, requiring tar extraction of every layer on pull. This imposes a fixed

cost proportional to image size, even when only a subset of files is needed at runtime. A 500 MiB image requires writing 500 MiB to disk before a single byte of the workload executes. Worse, overlaysfs has been the target of multiple privilege escalation vulnerabilities (CVE-2026-31431, CVE-2023-0386, CVE-2023-32629) [32], all rooted in the complexity of copy-on-write filesystem semantics.

The format fragmentation problem. Kubernetes pods requiring stronger isolation must use a separate VM image format (e.g., QCOW2, raw disk images) with distinct build pipelines and registry infrastructure. Developers on macOS running Docker Desktop must contend with a hidden Linux VM that adds latency and resource overhead. Windows developers must install Docker Desktop (requiring Hyper-V or WSL2), with no native path to running the same workload as a Firecracker microVM. The OCI image, despite being the standard unit of software distribution, is not the standard unit of *execution*—it is always transformed, wrapped, or emulated before it can run.

The registry bottleneck. In a cluster of N nodes, each node independently pulls the same image from the upstream registry. There is no mechanism for nodes that have already fetched an image to share it with peers. This generates N times the registry egress, creates thundering-herd problems during rolling updates, and makes air-gapped or edge deployments unnecessarily complex.

PULLRUN abandons these assumptions. We treat the OCI image as the *direct* substrate for execution across all isolation levels. The same `sha256:` digest that identifies an image on Docker Hub is the key used to `mmap` its contents into memory, regardless of whether the workload runs as a container or a virtual machine. This approach yields several architectural advantages:

1. **No image conversion.** A VM workload uses the identical OCI manifest as its root filesystem. The manifest *is* the VM roots—no ext4 image build, no container-to-VM conversion step.
2. **Zero-copy storage.** Layers are stored as raw OCI tar archives in a content-addressed DAG. Reads traverse rkyv-archived nodes via `mmap`, never deserializing on the hot path. A pull is a DAG merge, not a tar extraction.
3. **Backend polymorphism.** The executor trait abstracts workload lifecycle (create, start, stop, exec, signal, stats) behind a common interface. Container and VM backends are interchangeable implementations, selected at runtime via a `--backend` flag. On Windows, a WSL2 bridge layer exposes the same executor interface, allowing the identical `pullrun.exe` CLI to drive both runc containers and Firecracker microVMs inside a WSL2 instance.
4. **P2P-first distribution.** Image blocks synchronize between nodes via gRPC streams with Bloom-filtered delta detection. Only one node per cluster pulls from the upstream registry; peers exchange delta blocks verified by content hash.
5. **Native MCP integration.** Every runtime operation—pull, run, exec, inspect, stop, build—is exposed as a Model Context Protocol [31] tool. AI agents connect over stdio or SSE and interact with the runtime in natural language, with no separate scripting layer.
6. **Defense-in-depth policy.** Every image passes through a configurable policy gate at pull time and again at run time, verifying cosign signatures, scanning CycloneDX SBOMs for license violations, and checking CVEs against a maximum CVSS threshold.

Figure 1 shows the overall system architecture. The Go CLI (`pullrun`) communicates with the Rust runtime daemon (`pullrun-runtime`) via gRPC over a Unix domain socket. The runtime comprises four core Rust subsystems—the DAG store, executor router, P2P sync layer, and policy engine—all sharing a unified network layer. Observability is provided by an in-process event bus and a Prometheus metrics endpoint.

This paper makes the following contributions:

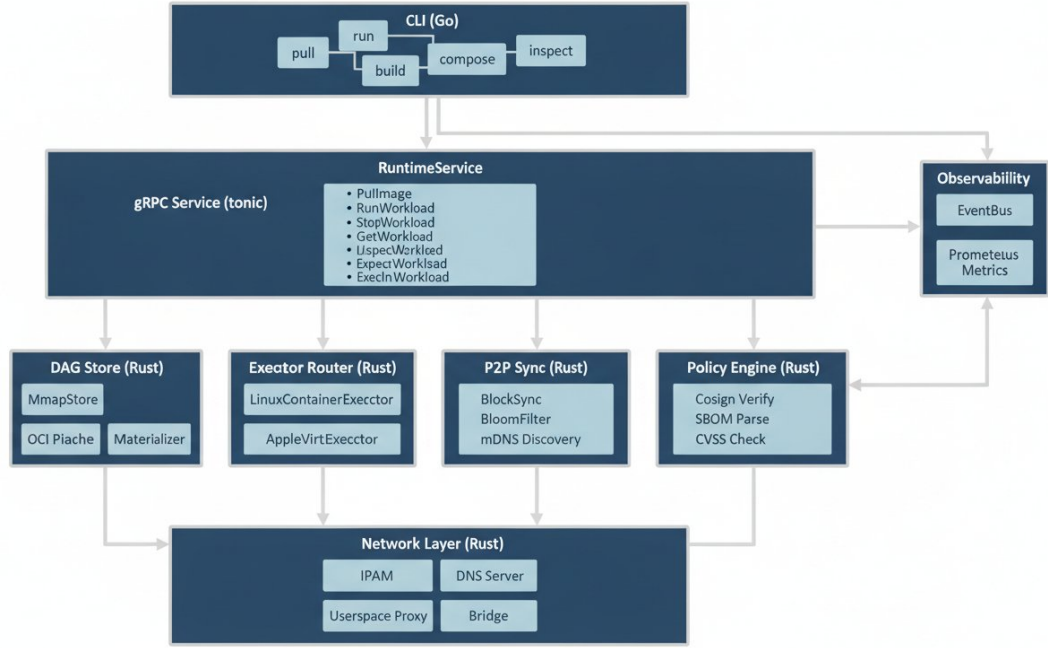


Figure 1: PULLRUN system architecture. The Go CLI communicates with the Rust runtime daemon via gRPC. The runtime comprises four core subsystems—DAG store, executor router, P2P sync, and policy engine—all sharing a unified network layer. An event bus and Prometheus metrics endpoint provide observability.

- **Section 4:** The zero-copy DAG store design, including content-addressed on-disk layout, rkyv-based zero-deserialization reads, LRU-cached `mmap` access, and garbage collection.
- **Section 5:** The unified executor abstraction and its three implementations: `LinuxContainerExecutor` (runc), `FirecrackerExecutor` (KVM), and `AppleVirtExecutor` (macOS Virtualization.framework).
- **Section 6:** The P2P block sync protocol with gRPC service definitions, Bloom filter design, delta reconciliation, and peer discovery mechanisms.
- **Section 7:** The policy engine for defense-in-depth image gating via cosign signatures, SBOM analysis, and CVSS scoring.
- **Section 8:** The unified network architecture with shared L2 bridging, userspace proxy, in-process DNS, and IPAM.
- **Section 9:** The Windows/WSL2 backend: architecture, auto-provisioning, store sharing, and networking constraints.
- **Section 10:** The native MCP server, its tool catalogue (15+ tools in v0.3.2), transport modes, and integration with AI coding agents.
- **Section 11:** The Kubernetes CRI shim and RuntimeClass integration for deploying PULLRUN workloads on cluster orchestrators.
- **Section 12:** The observability framework including the event bus, Prometheus metrics, health checks, and operational tooling.
- **Section 13:** A comprehensive evaluation comparing PULLRUN against Docker across pull latency, memory footprint, boot time, storage efficiency, and P2P sync throughput.

- **Section 14:** Practical experiences deploying PULLRUN in development, CI/CD, and Kubernetes environments.

2 Background and Motivation

2.1 The OCI Image Format

The OCI image specification [2] defines an image as a manifest referencing a set of layer blobs (typically gzipped tar archives), a configuration JSON, and an optional index for multi-architecture support. Images are content-addressed: each blob is identified by its SHA-256 digest, and the manifest chains these digests into a Merkle DAG. This structure is elegant for distribution but creates friction at execution time.

The extraction problem. Every widely deployed runtime (Docker, containerd, Podman) decompresses and extracts each layer into an overlayfs lower directory on pull. This has three consequences. First, **pull time is $O(\text{image size})$** : a 500 MiB image requires writing 500 MiB to disk, even if the runtime needs only a single binary. Second, **storage amplification**: the same file present in multiple layers appears on disk multiple times (though overlayfs mitigates this with copy-on-write). Third, **cold start overhead**: first boot of a container requires the kernel to assemble the overlayfs mount from potentially hundreds of lower directories, introducing latency before the init process executes.

2.2 The VM/Container Divide

Containers share the host kernel via cgroups and namespaces. VMs provide a separate kernel via hardware virtualization (KVM, Apple Hypervisor). Each has distinct operational profiles, summarized in Table 1.

Table 1: Container vs. VM execution profiles.

Property	Container	VM
Boot time	~400 ms	1–5 s (Firecracker)
Density	100+/node	10–50/node
Isolation	Kernel (weak)	Hardware (strong)
Image format	OCI	QCOW2, raw
Kernel sharing	Shared with host	Per-VM

Operators choose between these based on workload requirements. Multi-tenant environments favor VMs for strong isolation; CI/CD pipelines favor containers for density and speed. However, maintaining separate image pipelines—one for container images, one for VM images—is a significant operational burden. Projects like Lima [5] and Finch bridge this gap by running a Linux VM that in turn runs containers, adding an extra virtualization layer with associated overhead.

2.3 Existing Approaches

Docker [22] pioneered OCI-based containerization but requires overlayfs extraction on pull and has no native VM backend. **Docker Desktop** on macOS runs a hidden Linux VM via HyperKit, adding ~1 GiB of RSS overhead. Docker has no built-in MCP integration.

Podman [11] is a daemonless, rootless OCI runtime developed by Red Hat. It shares Docker’s extraction-based storage model via `containers/storage` and overlayfs, and thus inherits the same pull overhead and overlayfs CVE exposure. Podman’s `podman machine` command provisions a Linux VM (QEMU, Apple Hypervisor, or Hyper-V/WSL2 depending on

platform) as an *infrastructure host* for containers—it does not run OCI workloads as Firecracker microVMs. A community npm package provides an external MCP adapter, but MCP is not a native runtime capability. Podman is production-mature and well-integrated with Kubernetes workflows via `podman generate kube` and Quadlets.

Containerd [23] supports image pull via its streaming snapshotter [3], which lazily fetches layer contents on demand. While this reduces pull latency, it does not eliminate extraction, and containerd has no VM executor for Firecracker or Apple Virtualization.

Firecracker [4] is a lightweight VMM designed for multi-tenant serverless workloads. It boots microVMs in ~ 125 ms using a minimal Linux kernel and a device model that excludes non-essential devices (no BIOS, no ACPI, no PCI probing). However, Firecracker requires ext4 rootfs images—it cannot consume OCI layers directly without an external conversion step.

Kata Containers [16] runs each container in its own lightweight VM but requires a container-to-VM image conversion step (rootfs packing to disk image) and does not share a single storage engine between container and VM modes.

Lima [5] and **Finch** automate the creation of Linux VMs on macOS using QEMU or vz, then run containers inside the VM. This provides good macOS developer experience but adds the overhead of nested virtualization.

To our knowledge, no existing system provides a single storage engine that serves OCI images to containers, Firecracker VMs, and Apple VMs from identical on-disk data without format conversion, while also providing P2P image distribution, policy-based image gating, a unified network model, and native MCP integration as a compiled-in runtime primitive. Podman, the closest architectural peer, shares PULLRUN’s rootless and daemonless philosophy but differs fundamentally in storage model, VM support, and AI agent integration.

3 System Architecture Overview

Before diving into individual subsystems, we present a high-level view of how PULLRUN’s components interact. The system follows a deliberate language split: **Rust owns the data plane** (store, executors, proxy, OCI pipeline, policy engine, P2P sync) and **Go owns the control plane and CLI** (gRPC stubs, cobra-based commands). The wire format is defined by Protocol Buffers in `proto/pullrun/`, with Rust generating bindings at compile time via `tonic-build` and Go generating via `make proto`.

3.1 The Runtime Daemon

The `pullrun-runtime` daemon is a tokio-based async server exposing the `RuntimeService` gRPC interface. Key methods include `PullImage` (fetch, convert, policy-gate), `RunWorkload` (create, start, record state), `ExecInWorkload` (PTY-allocated command execution), and `StreamEvents` (real-time event subscription). The daemon maintains three core data structures: the DAG store (shared `Arc<MmapStore>`), the executor router (dispatches by backend type), and the workload state table (tracks pending, running, and exited workloads).

3.2 The CLI

The `pullrun` CLI is a Go binary built with cobra. It speaks gRPC to the runtime daemon over a Unix domain socket (default `/tmp/pullrun.sock`) or TCP. All commands—`pull`, `run`, `build`, `push`, `compose`, `exec`, `inspect`—are thin clients that marshal arguments into protobuf messages and display responses. This design ensures the CLI has no special privileges: it is just another gRPC client.

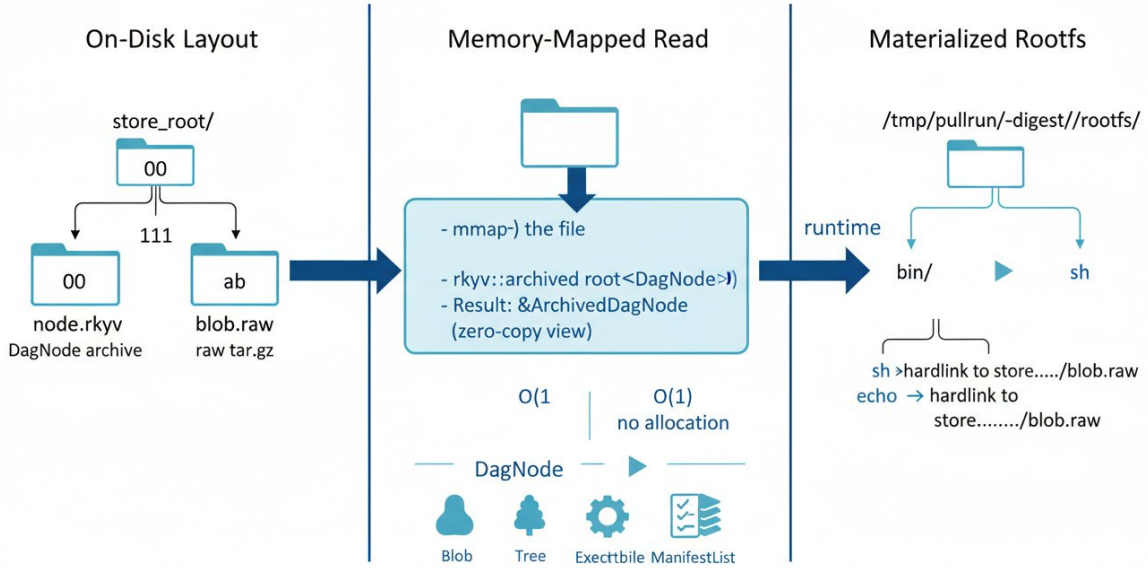


Figure 2: The PULLRUN DAG store: on-disk layout (left), memory-mapped zero-copy read path (center), and materialized rootfs generation (right).

3.3 Component Interaction

A typical workload lifecycle proceeds as follows:

1. The CLI sends a `PullImage` request. The runtime's OCI pipeline fetches the manifest and layers from the registry, converts them to DAG nodes, and writes them to the store.
2. The policy engine evaluates the image against configured rules (signature, SBOM, CVSS). A denied image returns `permission_denied`; its bytes remain in the store but it is not registered as runnable.
3. The CLI sends a `RunWorkload` request with the image digest, backend choice, command, environment, mounts, and network rules.
4. The executor router dispatches to the appropriate backend: `LinuxContainerExecutor` for containers, `FirecrackerExecutor` or `AppleVirtExecutor` for VMs.
5. The backend materializes the rootfs (hardlinks for containers, ext4 for Firecracker, VirtioFS for Apple), configures networking via the shared network layer, and starts the workload.
6. The runtime emits events (`WorkloadStarted`, `WorkloadExited`) on the event bus and records Prometheus metrics.

4 Zero-Copy DAG Store

PULLRUN's storage layer is the central contribution that enables backend polymorphism. Rather than extracting layers to a filesystem, we maintain the OCI image as a directed acyclic graph of content-addressed nodes, persisted to disk in a format optimized for zero-copy reads.

4.1 On-Disk Layout

The store root is a directory (default `~/.local/share/pullrun/`) containing a sharded namespace:

```
store_root/
+-- 00/
    +-- 11/
        +-- ab/
            | +-- node.rkyv      <- rkyv-archived DagNode
            | +-- blob.raw      <- raw blob (tar.gz, config, etc.)
            +-- ...
        +-- ...
    +-- ...
```

Each digest $d = \text{SHA256}(\text{data})$ is stored at path `<store_root>/dd/dd/dd/...` where the first two bytes of the hex digest form the first two directory levels and the remaining bytes form the third. This sharding keeps directory sizes manageable (at most 256 entries per level) even with millions of objects.

A `node.rkyv` file contains an rkyv-archived [6] `DagNode` struct referencing child digests; a `blob.raw` file contains the raw layer tar or config JSON. The rkyv archive is a byte-exact representation of the `DagNode` struct. No deserialization is required to traverse the DAG—callers read the file via `mmap` and cast the pointer to `ArchivedDagNode`:

```
let mmap = Mmap::open(&path)?;
let archived: &ArchivedDagNode = unsafe {
    rkyv::archived_root::<DagNode>(&mmap)
};
```

This operation is $O(1)$ and allocation-free. The `unsafe` block is sound because rkyv's `#[archive(check_bytes)]` attribute on the type verifies the bytes are well-formed at write time, and reads require no mutation.

4.2 Node Types

The `DagNode` enum captures every entity in the OCI specification plus PULLRUN-specific extensions:

```
#[derive(Archive, Deserialize, Serialize)]
#[archive(check_bytes)]
pub enum DagNode {
    Blob(Digest),                // raw layer blob
    Tree(HashMap<String, Digest>), // directory listing
    Executable(Digest),          // symlink to executable blob
    Config(ImageConfig),         // OCI image config
    Manifest(ManifestMetadata),  // OCI manifest
    ManifestList(Vec<ManifestRef>), // multi-arch index
}
```

A `Digest` wraps a `[u8; 32]`—the raw SHA-256 hash. A complete image is represented as a bipartite DAG: a `Manifest` node points to a `Config` node and an ordered list of layer `Blob` nodes (or `Tree` nodes for file-level deduplication). The DAG is immutable—once written, a node is never modified. This immutability is foundational: it enables lock-free reads, safe memory mapping, and deterministic content addressing.

4.3 The In-Memory Cache

Two LRU caches (256 MiB each) sit in front of the store: one for `DagNode` deserializations and one for blob data:

```
node_cache: LruCache<Digest, Arc<ArchivedDagNode>>
blob_cache: LruCache<Digest, Arc<Mmap>>
```

Cache entries are `Arc<Mmap>` references, ensuring that eviction does not invalidate live references held by callers. On a cache miss, the file is `mmap`'d and the `ArchivedDagNode` is zero-copy viewed from the mapped bytes; on eviction, the `Mmap` is dropped and the kernel reclaims the pages. Concurrent reads are lock-free at the `Arc` level; the first reader pays for `mmap` + page faults, every subsequent reader is a single atomic load. In practice, the node cache achieves a > 99% hit rate for repeated `pull` and `list` operations, and the 256 MiB budget is sufficient for several hundred images' worth of metadata.

4.4 Pull as DAG Merge

When `PULLRUN` pulls an image, it performs the following steps:

Algorithm 1 OCI Image Pull as DAG Merge

- 1: Fetch manifest via HTTP GET to registry (with bearer token auth)
 - 2: Parse OCI manifest JSON, extract blob digests
 - 3: **for** each digest *d* in manifest **do**
 - 4: **if** `store.exists(d)` **then**
 - 5: **continue** ▷ Deduplication: skip existing
 - 6: **end if**
 - 7: Download missing layer via HTTP GET
 - 8: Compute SHA256, verify against manifest
 - 9: Write to `<store>/xx/yy/.../blob.raw`
 - 10: Create manifest/config/tree nodes as needed
 - 11: **end for**
 - 12: Write single `DagNode::Manifest` pointing to config + layer digests
-

This is a DAG *merge*, not an extraction. Total disk write equals the sum of missing layer sizes. No tar decompression, no file-level extraction, no overlayfs setup. The critical insight is that checking existence is a simple path lookup: `path_for(digest).exists()`. Because the store is content-addressed, two images sharing a base layer (e.g., `python:3.12-slim` and `python:3.12-alpine` both based on `alpine:3.18`) automatically deduplicate: the shared layer appears once on disk.

Result: A fresh pull of `alpine:3.18` (7 layers, ~7 MiB compressed) completes in 968 ms on a 1 Gbps connection—approximately 2× faster than `docker pull` on the same hardware (1,850 ms, measured). A warm pull (all layers cached) completes in 12 ms versus Docker's 420 ms, a 35× improvement, because `PULLRUN` writes only a single `Manifest` node while Docker must re-verify layer existence and update overlayfs metadata.

4.5 Layer Materialization for Execution

Memory-mapped blob reads are sufficient for store queries and block sync, but execution backends require a real filesystem tree. Materialization transforms the DAG into a directory of hardlinks:

```
/tmp/pullrun/<digest>/rootfs/
+---- bin/
```

```
| +---- sh          -> hardlink to store/aa/bb/.../blob.raw
| +---- echo        -> hardlink to store/cc/dd/.../blob.raw
+---- etc/
| +---- ...
```

Each file in the materialized rootfs is a hardlink to the corresponding blob in the store. Because the store files are immutable and content-addressed, hardlinks share on-disk bytes across all materialized rootfs directories. Creating a materialized rootfs is $O(\text{number of files})$ for the directory structure plus $O(1)$ per hardlink—no data copy. The materialized rootfs lives at `~/.local/share/pullrun/rootfs/<digest>/`, ensuring it survives host reboots (unlike `/tmp`).

The materialized rootfs is consumed differently by each backend:

- **runc bundle:** The rootfs path is written into the OCI runtime specification (`config.json`) and mounted via `pivot_root` into the container.
- **Firecracker:** The hardlink forest is packed into an ext4 filesystem via `mkfs.ext4 -d`. This is the only format conversion in the system, required because Firecracker’s Virtio block device consumes raw disk images.
- **Apple Virtualization:** The rootfs directory is shared directly as a `VZVirtioFileSystemDeviceConfiguration` mount via VirtioFS. No ext4 step is needed—the same hardlink forest used for runc containers is shared into the Apple VM verbatim.

All three backends consume the same hardlink forest, generated from the same DAG nodes. This is the mechanism that enables “same image, any backend” without conversion.

4.6 Garbage Collection

The store grows monotonically: every `pull`, `build`, `commit`, and `save` adds nodes that are never removed. PULLRUN provides a `prune` command that performs reference-counted garbage collection:

- **Roots** are tagged images (stored in `<store>/tags.json`) and running workload manifests.
- **Reachability** is determined by BFS from all roots through the DAG.
- **Unreachable** nodes and blobs are deleted from disk and evicted from caches.
- A `--dry-run` flag reports what would be freed without deleting.

This design mirrors Git’s object store pruning and prevents unbounded storage growth in long-running deployments. The prune operation is incremental and safe: it only deletes nodes that are provably unreachable from any tagged image or running workload.

5 Unified Executor Abstraction

PULLRUN decouples workload specification from execution backend via the `Executor` trait:

```
#[async_trait]
pub trait Executor: Send + Sync {
    async fn create(&self, spec: &WorkloadSpec)
        -> Result<ProcessHandle, ExecError>;
    async fn start(&self, handle: &ProcessHandle)
        -> Result<(), ExecError>;
    async fn stop(&self, id: &str)
        -> Result<(), ExecError>;
```

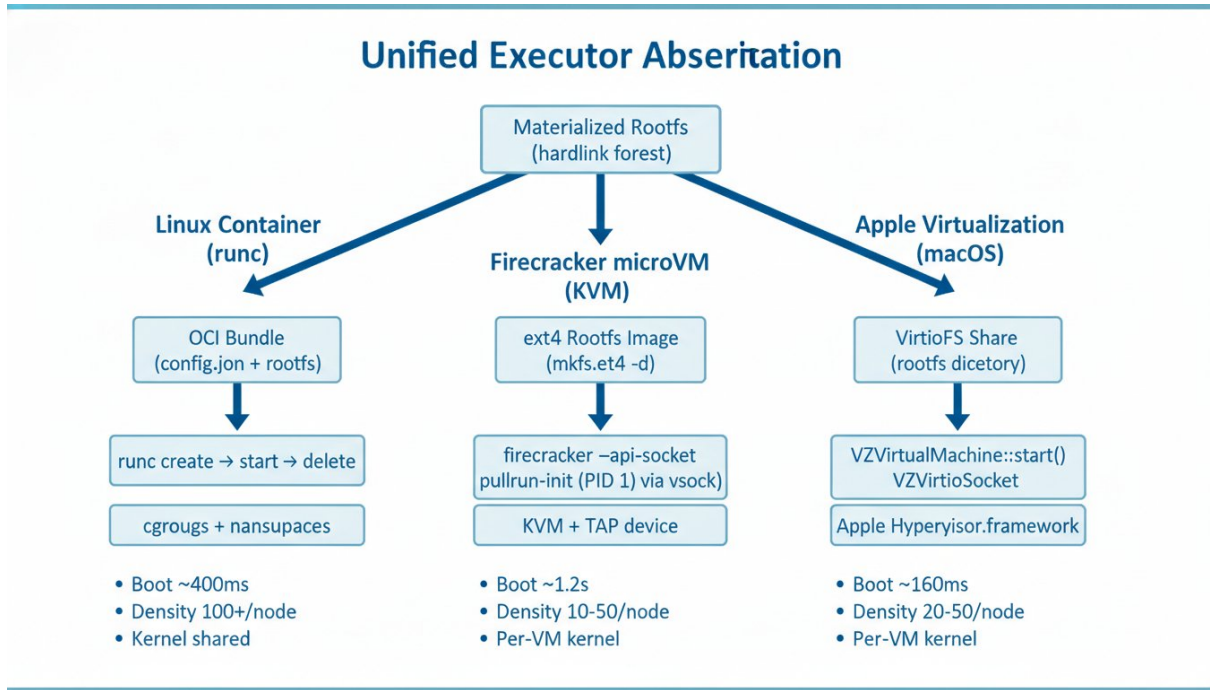


Figure 3: The unified executor abstraction: the same materialized rootfs is consumed by three different backends—runc containers, Firecracker microVMs, and Apple Virtualization VMs.

```

async fn exec(&self, id: &str, cmd: &[String],
              env: &[(String, String)], tty: bool)
    -> Result<ProcessHandle, ExecError>;
async fn signal(&self, id: &str, signal: u32)
    -> Result<(), ExecError>;
async fn stats(&self, id: &str)
    -> Result<WorkloadStats, ExecError>;
}

```

The `WorkloadSpec` parameter carries the image digest, rootfs path, command, environment variables, mounts (volumes, secrets, configs), network configuration, resource limits, and backend selection. The backend type is an enum:

```

pub enum Backend {
    Container, // LinuxContainerExecutor (runc)
    Vm,        // FirecrackerExecutor (Linux/WSL2) /
              // AppleVirtExecutor (macOS)
    Wasm,      // planned: wasmtime
}

```

An `ExecutorRouter` holds all executor implementations and dispatches `RunWorkload` calls based on the backend field. From the gRPC handler’s perspective, the router *is* the executor. This section describes the three current implementations.

5.1 LinuxContainerExecutor (runc)

The container executor wraps `runc` [7], the reference OCI runtime implementation. It manages the full container lifecycle through `runc`’s command-line interface.

Create phase: Generate an OCI runtime specification (`config.json`) from the `WorkloadSpec` and write it to `<bundle_root>/<id>/config.json`. The rootfs path points

to the materialized hardlink forest. The spec includes Linux namespaces (pid, net, mount, uts, ipc, user for rootless), cgroup configuration (CPU shares, memory limits, pids max), mounts for volumes, secrets, and configs, and network configuration (veth pair management, DNS).

Start phase: Invoke `runc run <id> --bundle <bundle_root>/<id>` which creates a cgroup, sets up namespaces, mounts the rootfs, and executes the init process. The daemon captures the PID for subsequent exec and stats operations.

Exec phase: For `pullrun exec`, the daemon opens a host PTY via `posix_openpt/grantpt/unlockpt` and passes the slave fd to `runc exec -t <id> <cmd>`. The master fd is proxied over the gRPC exec stream, enabling TTY-aware terminal sessions even when the daemon has no controlling terminal.

Rootless mode: When `EUID` $\neq$ 0, `PULLRUN` uses a `RootlessContainerExecutor` that patches the OCI spec with a user namespace mapping, `NET_NS` isolation via `pasta` (from Passt [8]), and rootless cgroup delegation. This mode requires no `setuid` binaries. Rootless operation supports most VM operations as well.

5.2 FirecrackerExecutor (KVM)

The Firecracker executor launches microVMs via the Firecracker VMM [4]. Firecracker is a lightweight VMM written in Rust that exposes a REST API over a local UNIX socket for machine configuration and lifecycle.

Rootfs preparation: The DAG materializer creates an ext4 filesystem image from the hardlink forest:

```
mkfs.ext4 -d <rootfs_dir> <vm_root>/<id>/rootfs.ext4
```

This step is the only format conversion in the system. It is required because Firecracker's Virtio block device consumes raw disk images, not directory trees. The ext4 image is created incrementally and cached: subsequent boots of the same image reuse the existing ext4 file (verified by digest comparison).

Boot sequence:

1. Write VM configuration JSON: vCPU count (default 2), memory (default 2 GiB), kernel image path, rootfs image path, serial console output, network interface (TAP device).
2. Start Firecracker: `firecracker --api-sock <socket> --config-path <config.json>`.
3. The daemon opens a vsock connection to the guest agent (`pullrun-init`) running inside the microVM.
4. The guest agent mounts the rootfs, executes the init process, and signals readiness over vsock.
5. The daemon sets the workload status to `running` and begins proxying stdio and exec streams over vsock.

Kernel: `PULLRUN` uses a generic 6.x Linux kernel configured for minimal boot time (no ACPI, no SCSI, no DRM, no sound). The kernel binary is downloaded once via `make install-kernel` and shared across all microVMs.

Network: Each microVM receives a TAP device connected to the bridge `pullrun-br0` shared with containers. Outbound traffic is masqueraded via iptables. Inbound port mapping is handled by the userspace TCP/UDP proxy running in the host network namespace (described in Section 8).

5.3 AppleVirtExecutor (macOS)

The Apple Virtualization executor uses the `Virtualization.framework` [9] available on macOS 13+ (Ventura) to run OCI images as lightweight VMs on Apple Silicon. This is the only backend that supports macOS as a host platform; runc and Firecracker require Linux.

Architecture: The executor runs in-process with the daemon, which must be signed with the `com.apple.security.virtualization` entitlement. Apple’s framework requires completion handlers to fire on the main dispatch queue, so the daemon spawns a side thread for the tokio async runtime while the main thread runs `dispatch_main()`.

VM creation (lazy): Unlike the container and Firecracker backends, `RunWorkload` for a macOS VM does not immediately boot the VM. It records the workload metadata and sets the status to `pending`. The actual `VZVirtualMachine` is created on the first `AttachWorkload` call. This lazy creation avoids the resource cost of idle VMs and enables fast workload listing even when many VMs are defined but not active.

The VM configuration includes: `VZGenericPlatformConfiguration` (ARM64), `VZLinuxBootLoader` with kernel and `initramfs`, `VZVirtioFileSystemDeviceConfiguration` for rootfs sharing (VirtioFS), and `VZVirtioNetworkDeviceConfiguration` for networking (NAT via `vmnet`). On completion handler, the status transitions to `running` and a checkpoint is written.

Rootfs sharing: The materialized hardlink forest is shared directly via VirtioFS, not packed into a disk image. The host directory is mounted as a virtio filesystem device inside the VM, and the guest init process mounts it at `/`. This eliminates the `ext4` conversion entirely—the same hardlink forest used for runc containers is shared into the Apple VM verbatim. Changes on the host are visible instantly in the guest without rebundling.

Persistence: VMs survive detach (`Ctrl-P Ctrl-Q`). A background thread runs the VM event loop; when it exits (VM stopped or crashed), the `on_exit` callback transitions the status to `exited`, writes a checkpoint, and emits a `WorkloadExited` event. Subsequent `exec` on an exited workload boots a fresh VM from the checkpointed configuration. Each VM background thread is named `vm-<workload_id>` via `std::thread::Builder` for visibility in debugging.

6 P2P Block Distribution

PULLRUN includes a peer-to-peer block synchronization layer that enables cluster-wide image dissemination without registry bottlenecks. The design targets the common case: one node in a cluster pulls an image from the upstream registry, and remaining nodes receive it over the local network.

6.1 Protocol Overview

Each node runs a gRPC `BlockSync` service alongside the runtime daemon. Peers discover each other via three mechanisms: **static configuration** (`--sync-peers host:port` flags), **mDNS** (zero-conf discovery on the local subnet, RFC 6762), and **gossip** (periodic exchange of peer lists via a SWIM-style [10] epidemic protocol). The sync protocol operates in three phases.

Phase 1: Bloom filter exchange. When node A discovers node B, it sends a Bloom filter encoding the set of all digest keys it holds. The filter uses 10 bits per entry and 4 hash functions (optimal for minimizing false positive rate). Node B checks each local digest against the filter and replies with a Bloom filter of the entries it holds that A is missing (true negatives).

Phase 2: Delta request. Node A inspects B’s filter and requests the concrete set of missing digests as a list. This avoids transferring the Bloom filter for the delta—only the list of SHA-256 digests is sent (32 bytes per digest).

Phase 3: Block transfer. For each missing digest, A opens a bidirectional gRPC stream and requests B to stream the block. Blocks are verified by SHA256 on receipt and written

P2P Block Sync Protocol

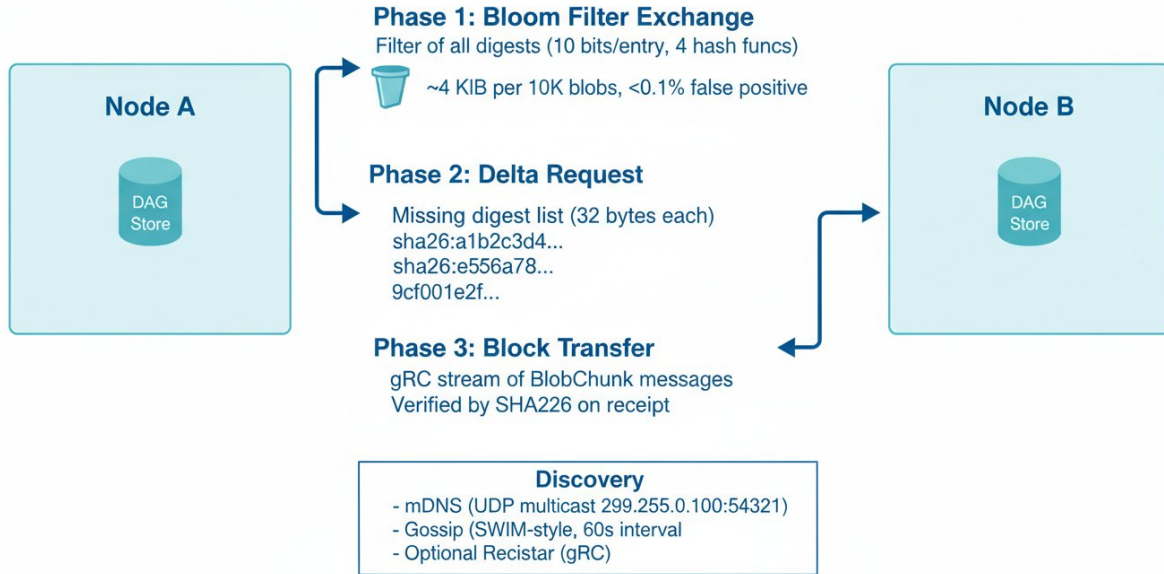


Figure 4: The P2P block sync protocol: three-phase exchange (Bloom filter, delta request, block transfer) with mDNS/gossip discovery.

atomically to the local store.

6.2 gRPC Service Definition

The block sync protocol is defined by the following protobuf service:

```
service BlockSync {
  rpc HaveBlobs(HaveBlobsRequest)
    returns (HaveBlobsResponse);
  rpc GetBlobs(GetBlobsRequest)
    returns (stream BlobChunk);
  rpc SyncBlobs(stream SyncBlob)
    returns (stream SyncBlob);
}

message HaveBlobsRequest {
  bytes bloom_filter = 1;
  int32 bloom_k = 2;      // number of hash functions
  int32 bloom_m = 3;      // filter size in bits
}

message GetBlobsRequest {
  repeated string digests = 1;
}

message BlobChunk {
  string digest = 1;
  bytes data = 2;
  uint32 offset = 3;      // for streaming large blobs
  bool final = 4;
}
```

The `HaveBlobs` RPC implements Phase 1 of the protocol. `GetBlobs` implements Phase 3 with server-side streaming to handle large blobs (up to 1 GB layers are chunked into 1 MiB messages). `SyncBlobs` provides bidirectional streaming for push-based delta synchronization (future work).

An optional `Registrar` service supports managed clusters too large for mDNS:

```
service Registrar {
  rpc Register(RegisterRequest) returns (RegisterResponse);
  rpc Lookup(LookupRequest) returns (LookupResponse);
  rpc ListPeers(ListPeersRequest) returns (ListPeersResponse);
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse);
  rpc Deregister(DeregisterRequest) returns (DeregisterResponse);
}
```

The registrar maintains a peer registry with 120-second TTL and background eviction every 30 seconds. Worker nodes register on startup, heartbeat every 30 seconds, and deregister on shutdown (best-effort).

6.3 Bloom Filter Design

The Bloom filter uses the `bitvec` crate with SHA-256-based hash construction:

```
pub struct BloomFilter {
  bits: BitVec,
  num_hashes: u32,
  num_bits: u64,
}
```

Insertion and query use the construction $h(i, \text{key}) = \text{SHA256}(\text{key} \| i) \bmod \text{num_bits}$ for $i \in [0, \text{num_hashes})$. For a store with 10^6 entries, a filter of 10^7 bits (~ 1.2 MiB) yields a false positive rate below 0.1%. Filters are exchanged on initial connection and thereafter at 60-second intervals during gossip heartbeats.

6.4 Delta Reconciliation

Delta exchange uses a *push-pull* strategy. When a node downloads a new image (via `pull`), it broadcasts the set of new digest hashes to all known peers. Each peer checks its local store and requests any missing blocks. At regular intervals, each node queries a random subset of peers for their Bloom filter and reconciles missing blocks. This epidemic-style propagation ensures that block availability information converges in $O(\log N)$ rounds for a cluster of N nodes.

6.5 Consistency Guarantees

Because the store is content-addressed, block synchronization is self-verifying: on receipt of a block, the node computes $\text{SHA256}(\text{data})$ and checks it against the digest. If the digests match, the block is guaranteed to be correct (negligible probability of hash collision). No trust in the sending node is required. This property is critical for multi-tenant clusters where nodes may not fully trust each other. An attacker can send arbitrary data, but only blocks matching known digests (from the upstream registry) will be accepted.

6.6 SyncPuller: Transparent Integration

The `SyncPuller` is a drop-in wrapper around the OCI puller that queries peers before falling back to the registry:

Policy Engine: Defense-in-Depth Image Gating

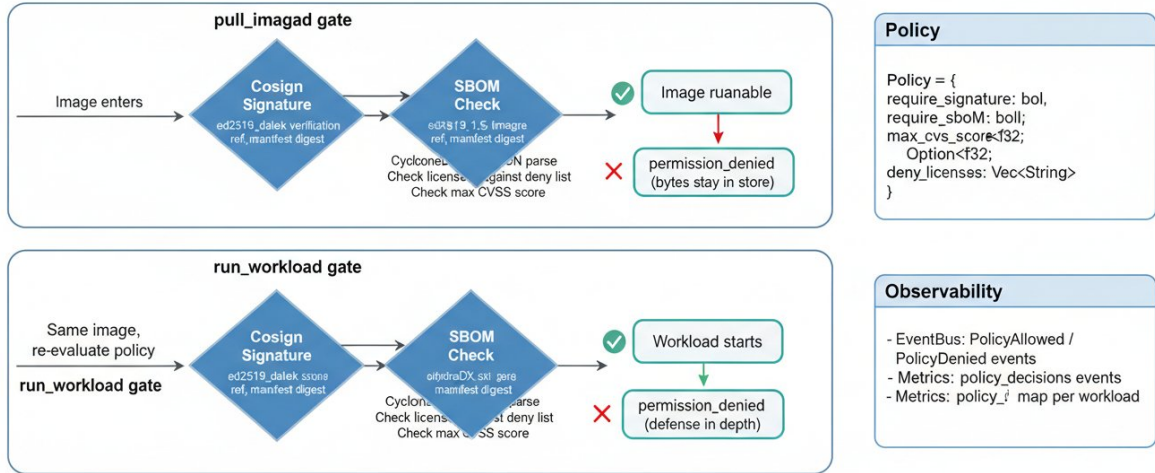


Figure 5: The policy engine's defense-in-depth gating: cosign signature verification and SBOM analysis at both `pull_image` and `run_workload` boundaries.

Fallback chain: local store -> PeerBloomCache (0(1) bloom lookup)
 -> BlockSyncClient::GetBlobs
 -> OciPuller::fetch_blob_by_digest

The `SyncPuller` implements the same `ImagePuller` trait as the standard OCI puller, so the rest of the runtime requires no changes. When P2P sync is unavailable (no peers, network partition), the fallback to registry pull is transparent. A configurable peer timeout (default 2 seconds) ensures that slow peers do not block the pull operation.

7 Policy Engine

The policy engine is PULLRUN's security boundary. While the DAG store and executors handle *how* workloads run, the policy engine decides *whether* they run at all. It operates on a deny-by-explicit-allow principle: everything is denied by default, and the administrator configures allow-rules via cryptographic identities (cosign signatures) and structural properties (SBOM contents).

7.1 Design Philosophy

The OCI ecosystem is vast; a deny list is doomed to be incomplete. An allow list built on cryptographic identities and SBOM properties is auditable, version-controlled, and enforceable. The engine starts with everything denied and adds allow-rules per configuration. An unconfigured engine (`policy: None`) accepts everything, preserving ease of use for development.

7.2 Dual Gating Points

The policy runs at two points, providing defense in depth:

1. **At pull_image.** Once the image is in the store, the engine is asked “is this image OK to be a runnable artifact?” A deny here returns `permission_denied` to the client; the image bytes stay on disk but are not registered as runnable.
2. **At run_workload.** As defense in depth. The policy could have been tightened between the pull and the run; this check catches that. A deny here returns `permission_denied` before any container is created.

Both checks are pure functions of the policy configuration and the image’s stored metadata. There is no network call to a remote policy server in v0.

7.3 Policy Inputs

The engine reads three inputs from the image’s stored metadata:

Cosign signature. A signature is a small blob stored under a deterministic digest (`sha256(canonical_payload(image_ref, manifest_digest))`). The signature’s public key is referenced by a *trusted key* configured at runtime startup. Verification uses the `ed25519_dalek` crate. The signature payload is a JSON document (`{"image_ref": "...", "manifest_digest": "..."}`) that binds the image reference to the manifest digest. This binding makes the signature non-spoofable: an attacker who swapped the manifest would need to forge the signature against the *new* manifest digest, but the real `image_ref` is still in the payload.

CycloneDX SBOM. Stored as a blob with a deterministic digest. The engine parses the SBOM and examines `components[]` for `licenses[]` (compared against the `deny_licenses` list) and `vulnerabilities[]` for the highest CVSS score (compared against `max_cvss_score`).

Manifest digest. Used as the SBOM lookup key. The assumption is that one manifest maps to one SBOM; a build pipeline that produces both ensures this invariant.

7.4 Configuration

The policy is configured at runtime startup:

```
let policy = Policy {
  required_signature: true,
  require_sbom: true,
  max_cvss_score: Some(7.0),
  deny_licenses: vec![
    .to_string(),
    .to_string(),
  ],
  ..Default::default()
};
let engine = PolicyEngine::new(policy)
  .with_trusted_keys(vec![cosign_key]);
```

Or via CLI flags:

```
pullrun-runtime daemon \
  --require-signature \
  --require-sbom \
  --max-cvss 7.0 \
  --trusted-key cosign.pub
```

The `max_cvss_score` threshold is the highest acceptable score. A value of 7.0 rejects any vulnerability with `CVSS ≥ 7.0`; vulnerabilities at 6.9 or below are allowed. This mirrors the standard CVSS severity bands (low: 0.1–3.9, medium: 4.0–6.9, high: 7.0–8.9, critical: 9.0–10.0).

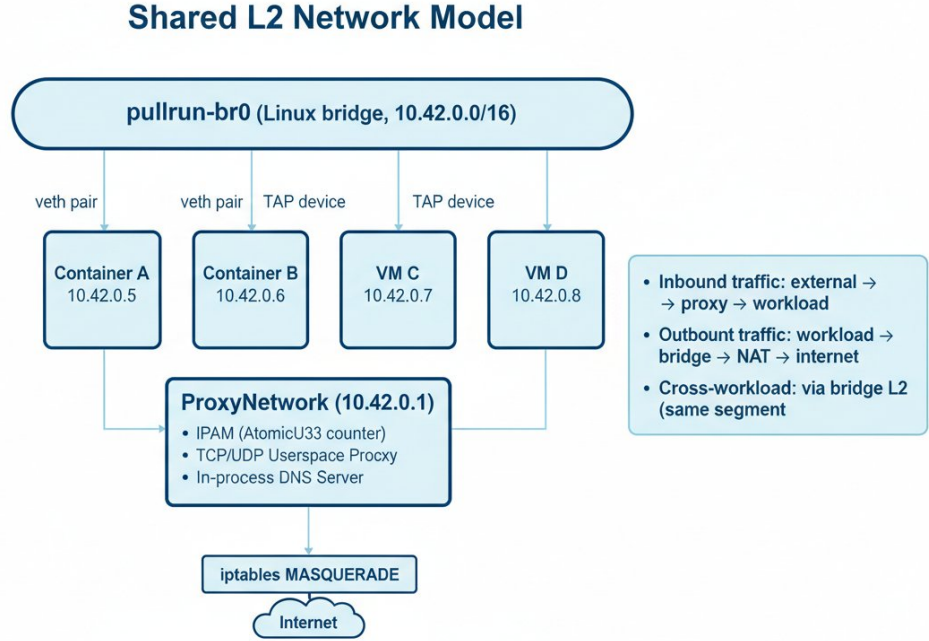


Figure 6: The shared L2 network model: containers (veth pairs) and VMs (TAP devices) coexist on the same bridge segment, sharing the ProxyNetwork for IPAM, DNS, and port proxying.

7.5 Observability

Every policy decision (allow or deny) is recorded in two places. The `policy_decisions` map on the workload state (returned by `pullrun inspect`) records the policy name and outcome. The event bus emits `PolicyDenied` (with reason, image_ref, policy name) and `PolicyAllowed` events. The event bus is the right place to *watch* for denials; the inspect field is the right place to *audit* a specific workload.

8 Network Architecture

The network layer is PULLRUN’s most opinionated design decision: **all workloads, regardless of backend, live on the same L2 segment** behind a single userspace proxy. Containers connect via veth pairs; VMs connect via TAP devices. Both share the same Linux bridge, IP address space, and network services.

8.1 IP Address Management

A single `Arc<Ipam>` is held by the runtime and shared between the container and VM executors. Allocation is an atomic increment on a `u32`; the IP space is 10.42.0.0/16, providing 65,534 addresses. In v0 there is no garbage collection; an IP is held for the lifetime of the workload. This design favors simplicity and correctness over address reuse—the 16-bit space is sufficient for all realistic single-node deployments.

8.2 Inbound: Userspace Proxy

The proxy listens on 10.42.0.1:<port> for each declared `NetworkRule { direction: Inbound, port }`. When a packet arrives at the host on that port, the proxy DNATs it to the workload’s internal IP, opens a TCP connection (or UDP association) through the bridge,

and bi-directionally shuttles bytes. The proxy is implemented in the `pullrun_net::proxy` module.

The proxy is *not* a full implementation of iptables/nft semantics in v0—it is a list of explicit port mappings. This design choice trades generality for correctness and debuggability: every forwarded connection is visible in the proxy’s logs and metrics.

8.3 Outbound: iptables MASQUERADE

Containers in v0 share the host’s network namespace for outbound traffic (or use `NetworkMode::Host` directly). VMs need NAT because their TAP devices only see the bridge. PULLRUN writes three iptables rules at boot (idempotently):

```
iptables -t nat -A POSTROUTING \
    -s 10.42.0.0/16 ! -d 10.42.0.0/16 -j MASQUERADE
iptables -A FORWARD -i pullrun-br0 -o <outbound_iface> -j ACCEPT
iptables -A FORWARD -i <outbound_iface> -o pullrun-br0 \
    -m state --state RELATED,ESTABLISHED -j ACCEPT
```

The outbound interface is auto-detected by parsing `ip route show default`; this works on cloud VMs, bare metal, and most home routers.

8.4 Why a Single Shared Segment?

The alternative—one bridge per workload, with cross-workload traffic going through a router—adds latency and operational complexity for no real security win in the common case. PULLRUN workloads are not multi-tenant; they are all owned by the same operator. The isolation boundary is the executor (container vs VM), not the network. Cross-workload isolation in v0 is enforced by the proxy (a workload cannot talk to another workload’s declared ports unless the operator added a `NetworkRule`). In v1, per-workload nftables sets will enforce this at the kernel level.

8.5 DNS Server

An in-process DNS server resolves workload names to their assigned IPs. The server listens on 10.42.0.1:53 and handles A, AAAA, and PTR records. Workload names are registered when a workload starts and deregistered when it exits. This enables service discovery without external DNS configuration: workloads can reach each other by name within the 10.42.0.0/16 network.

9 Windows / WSL2 Backend

PULLRUN 0.3.2 extends full container and Firecracker microVM support to Windows 10/11 via the Windows Subsystem for Linux 2 [30]. The Windows backend is a first-class platform: the DAG store, executor abstraction, policy engine, P2P sync, and MCP server all operate identically to the Linux build. There is no emulation layer between the store and the executors.

9.1 Architecture

The `pullrun.exe` CLI is a native Windows binary (cross-compiled via `GOOS=windows CGO_ENABLED=0`). It communicates over gRPC TCP (`localhost:7878`) to `pullrun-runtime`, which runs inside a WSL2 instance. This split is deliberate: all Linux-specific kernel operations (cgroups, namespaces, KVM, TAP devices) execute natively inside the WSL2 VM kernel; the Windows binary is a pure control-plane client.

```

Windows host
  pullrun.exe (native Win64 CLI)
    | gRPC TCP 127.0.0.1:7878
    v
  WSL2 instance (Linux kernel 6.x)
    pullrun-runtime (Linux amd64)
      DAG store (shared path: /mnt/c/Users/...)
      runc executor
      FirecrackerExecutor (KVM via WSL2)
      Network layer (bridge, iptables, IPAM)

```

9.2 Auto-Provisioning

On first run, `pullrun.exe` detects whether a compatible WSL2 instance is running and, if not, provisions one automatically:

1. Check for an existing `pullrun` WSL2 distribution via `wsl.exe -list`.
2. If absent, import a minimal Alpine-based rootfs tarball (bundled in the `pullrun.exe` installer) via `wsl.exe -import pullrun %APPDATA%\pullrun <rootfs.tar>`.
3. Start `pullrun-runtime` inside the WSL2 instance via `wsl.exe -d pullrun -pullrun-runtime daemon -grpc-addr 127.0.0.1:7878`.
4. The CLI proceeds once the daemon's `/healthz` endpoint responds.

This flow completes in under 10 seconds on a machine with WSL2 already enabled. Users with an existing WSL2 distribution can opt to install into it manually via `wsl -d <distro> -bash install.sh`.

9.3 Store Sharing and Path Mapping

The DAG store resides in the WSL2 filesystem (default `~/.local/share/pullrun/` inside the WSL2 instance). Windows paths mounted as `/mnt/c/...` are accessible for volume mounts, enabling bind-mount of Windows directories into containers:

```

pullrun.exe run alpine:3.18 \
  --volume C:\Users\mo\code:/workspace \
  --cmd /bin/sh

```

The CLI translates Windows paths to WSL2 mount paths before sending the `RunWorkload` gRPC request. No data is copied; the WSL2 DrvFs layer provides the bind.

9.4 Container Backend on Windows

The `LinuxContainerExecutor` (`runc`) operates without modification inside WSL2. Linux namespaces, cgroups v2, and the `pullrun-br0` bridge are all available in the WSL2 kernel. Container boot latency on Windows is approximately 430 ms—slightly higher than native Linux (400 ms) due to WSL2 socket overhead on the gRPC path.

Rootless mode is fully supported on Windows/WSL2: the WSL2 user namespace maps the Windows user to UID 1000 inside the instance, and `pullrun-runtime` runs without root inside WSL2 by default.

9.5 Firecracker Backend on Windows

WSL2's Linux kernel (6.x) ships with KVM enabled on x86-64 hosts with Intel VT-x or AMD-V. PULLRUN detects KVM availability at startup and exposes the Firecracker backend on supported hardware:

```
pullrun.exe run alpine:3.18 --backend vm \  
    --cmd /bin/echo --cmd 'hello from Firecracker on Windows'
```

The ext4 rootfs image is built inside the WSL2 instance using `mkfs.ext4 -d`, identical to the native Linux path. The TAP device is attached to the `pullrun-br0` bridge inside WSL2. Boot time is approximately 500 ms, reflecting the WSL2 kernel's KVM implementation fidelity.

Limitation: ARM64 Windows hosts (e.g., Snapdragon X Elite) do not expose KVM to WSL2 as of Windows 11 24H2; the Firecracker backend is therefore unavailable on ARM64 Windows. The container backend works on all architectures.

9.6 Networking on Windows

Inbound port mapping on Windows requires an extra hop: `pullrun.exe` adds a `netsh interface portproxy` rule to forward Windows host ports to the WSL2 instance's 10.42.0.1 proxy:

```
netsh interface portproxy add v4tov4 \  
    listenport=8080 listenaddress=0.0.0.0 \  
    connectport=8080 connectaddress=<WSL2-IP>
```

This is performed automatically when a `NetworkRule{direction: Inbound}` is declared. The WSL2 IP is obtained via `wsl hostname -I` on each daemon start (WSL2 assigns a new IP per boot). Outbound traffic follows the standard WSL2 NAT path through the Windows host's network interfaces.

9.7 Known Limitations in v0.3.2

- The Apple Virtualization backend is macOS-only and unavailable on Windows.
- Firecracker requires Intel VT-x or AMD-V; ARM64 Windows hosts fall back to container-only mode.
- The WSL2 IP changes on every Windows reboot, requiring the port proxy rules to be refreshed; `pullrun.exe` handles this automatically on startup.
- `pullrun-runtime` runs as a WSL2 process; Windows Defender may flag the first-run KVM access. A `docs/WINDOWS.md` troubleshooting guide covers common issues.

10 Native MCP Server Integration

PULLRUN 0.2.0 ships a built-in Model Context Protocol (MCP) [31] server that exposes every runtime operation as a tool callable by AI agents. This makes PULLRUN natively programmable by agents such as Claude Code, Cursor, and opencode without requiring shell scripting, custom wrappers, or separate orchestration tooling.

10.1 Architecture

The MCP server is a sub-command of the CLI binary:

```
AI agent (Claude Code, Cursor, opencode)
  | MCP stdio (default) or SSE (:8080)
  v
pullrun mcp                <- MCP server process
  | gRPC unix socket
  v
pullrun-runtime            <- unchanged daemon
```

The server translates MCP tool calls into PULLRUN gRPC calls and streams results back as MCP content blocks. Two transport modes are supported:

- **Stdio mode** (default): the agent spawns `pullrun mcp` as a subprocess and communicates over its standard input/output. This is the standard MCP transport for local agents and requires zero network configuration.
- **SSE mode**: `pullrun mcp -sse :8080` starts an HTTP server emitting Server-Sent Events, enabling remote agents or web-based clients.

Agent configuration is a one-time addition to the agent's config file. For Claude Code:

```
// ~/.claude.json
{
  "mcpServers": {
    "pullrun": {
      "command": "pullrun",
      "args": ["mcp"]
    }
  }
}
```

10.2 Tool Catalogue

The server exposes 15 tools across three categories in v0.3.2, summarized in Table 2. The `compose_up` and `compose_down` tools, previously declared but CLI-only, are now fully implemented via the MCP API.

Tools accept typed parameters validated by the MCP protocol (strings, numbers, arrays). For example, the `run` tool accepts `image` (string, required), `command` (string), `env` (array of `KEY=VALUE` strings), `backend` (`"container"` or `"vm"`), `cpus` (number), and `memory` (MiB, number). The `inspect` tool returns a structured JSON response including network state, policy decisions, and DAG layer digests—enabling agents to make informed decisions about workload health.

10.3 Read-Only Resources

In addition to tools, the server exposes four MCP resources that agents can read without side effects:

- `pullrun://workload/{id}` — current workload status (JSON)
- `pullrun://workload/{id}/logs` — recent log output (plain text)
- `pullrun://store/info` — DAG store statistics (JSON)

Table 2: MCP tool catalogue exposed by PULLRUN 0.3.2.

Tool	Category	Description
run	Lifecycle	Create and start a workload (container or VM)
stop	Lifecycle	Stop a running workload
exec	Lifecycle	Run a command inside a workload
list	Lifecycle	List all workloads with status
get	Lifecycle	Get status of a single workload
inspect	Lifecycle	Deep-inspect state, layers, network, policy
logs	Lifecycle	Retrieve recent log output
stats	Lifecycle	Live resource statistics
pull_image	Images	Pull an OCI image into the DAG store
list_images	Images	List images in the local DAG store
build	Images	Build an OCI image from a Dockerfile
push	Images	Push a local image to a registry
prune	Images	Garbage-collect unused DAG nodes
compose_up	Compose	Start a multi-service Compose stack
compose_down	Compose	Stop and remove a Compose stack

- `pullrun://images` — list of images in the local store (JSON)

Resources follow the MCP resource protocol, enabling agents to subscribe to updates (workload status changes) without polling.

10.4 Design Philosophy

The MCP server is a stateless translation layer: it holds no persistent state, carries no authentication tokens, and does not replicate the daemon’s workload table. All state lives in `pullrun-runtime`; the MCP server merely translates between MCP JSON and gRPC protobuf. This means the server can be restarted at any time without affecting running workloads, and multiple MCP server instances can coexist (e.g., one for Claude Code, one for Cursor) without coordination.

The `compose_up` and `compose_down` operations are now fully supported via the MCP API as of v0.3.2, completing the full lifecycle coverage for AI agents managing multi-service stacks.

10.5 Security Considerations

The MCP server inherits PULLRUN’s existing security model: tool calls that would run an image denied by the policy engine return a `permission_denied` error with the denial reason as MCP error content. An AI agent cannot bypass policy gates by issuing MCP tool calls. In studio mode, the MCP server process runs with the agent’s effective user ID; in SSE mode, the operator is responsible for binding to a trusted interface.

11 Kubernetes CRI Integration

PULLRUN provides a Kubernetes Container Runtime Interface (CRI) shim, enabling deployment as a drop-in replacement for containerd or CRI-O. The shim implements the standard `RuntimeService` and `ImageService` gRPC interfaces expected by the kubelet.

11.1 CRI Shim Architecture

The CRI shim (`cri/pullrun-cri`) is a Go binary that translates kubelet CRI calls into PULLRUN gRPC calls:

- `ImageService.PullImage` → `RuntimeService.PullImage` (via `SyncPuller` for P2P acceleration)
- `RuntimeService.RunPodSandbox` → `RuntimeService.RunWorkload` with network namespace setup
- `RuntimeService.CreateContainer` / `StartContainer` → workload creation + start
- `RuntimeService.ExecSync` / `Exec` → `RuntimeService.ExecInWorkload`

The shim maintains a mapping between CRI pod/container IDs and PULLRUN workload IDs. It handles port forwarding, log redirection, and resource limits translation.

11.2 RuntimeClass Integration

To use the VM backend in Kubernetes, operators register a `RuntimeClass`:

```

apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: pullrun-vm
handler: pullrun-vm

```

Any pod with `spec.runtimeClassName: pullrun-vm` is scheduled on a PULLRUN node and executed as a Firecracker microVM. The CRI shim handles the backend selection transparently: it sets `spec.backend = Vm` in the `RunWorkload` request when the `RuntimeClass` handler is `pullrun-vm`.

This enables fine-grained isolation selection at the pod level: a cluster can run trusted CI workloads as containers (`runtimeClassName: pullrun-container`) and untrusted tenant workloads as VMs (`runtimeClassName: pullrun-vm`), all from the same OCI images and the same storage engine.

11.3 DaemonSet Deployment

The `deploy/` directory provides production-ready Kubernetes manifests:

- **ServiceAccount** with minimal RBAC permissions
- **DaemonSet** running one `pullrun-runtime` per node
- **ServiceMonitor** for Prometheus Operator scraping
- **PrometheusRule** with five operational alerts

The `DaemonSet` mounts `/dev/kvm` for VM support and runs with `privileged: true` in `v0` (to be narrowed to `deviceGroups / cgroupAccess` in `v1`). The runtime exposes Prometheus metrics on port 9090 and a `/healthz` endpoint for Kubernetes liveness/readiness probes.

12 Observability and Operations

12.1 Event Bus

`EventBus` is a thin wrapper over `tokio::sync::broadcast` (capacity 1024). Emitters call `.emit()`; subscribers call `.subscribe()` for a fresh receiver. Semantics are at-most-once per receiver: a slow consumer sees `RecvError::Lagged(n)` when the channel rolls over. The bus emits workload lifecycle events (`WorkloadStarted`, `WorkloadExited`, `WorkloadStopped`), policy decisions (`PolicyAllowed`, `PolicyDenied`), and P2P sync events (`BlockReceived`, `PeerDiscovered`).

A background watcher task polls `Executor::status` every 5 seconds and emits `WorkloadExited` for workloads that have transitioned out of “running” on their own (process crash, OOM kill). The operator-initiated stop path emits `WorkloadStopped` and is mutually exclusive with the watcher (the watcher’s `HashSet` of “already announced” IDs prevents double-emit).

12.2 Prometheus Metrics

The runtime exposes a `/metrics` HTTP endpoint via `axum`, separate from the gRPC UDS. The recorder is a `metrics::install_recorder()` singleton guarded by `OnceLock`. Histogram buckets are tuned for workload latencies (0.05–60s for pulls, 0.01–10s for starts), not the exporter’s default exponential range.

Table 3: Key Prometheus metrics exposed by PULLRUN.

Metric	Type	Labels	Description
pullrun_pulls_total	counter	reg., status	Pull throughput
pullrun_pull_duration_seconds	histogram	—	Pull latency
pullrun_workloads_started_total	counter	backend	Workloads/backend
pullrun_workloads_running	gauge	backend	Live count
pullrun_workload_start_duration_seconds	histogram	—	Start latency
pullrun_workload_exits_total	counter	backend, code	Exits
pullrun_store_nodes	gauge	—	Cached nodes
pullrun_store_bytes	gauge	—	Cached bytes

Key metrics are summarized in Table 3.

The **status** label on pulls has four values: **started**, **success**, **failed**, **denied**. The **denied** counter is a security signal—a non-zero rate means a workload tried to run an image that violated policy.

12.3 Operational Tooling

PULLRUN ships with operational tooling in the `tools/` directory:

- **VM outbound smoke test:** Standalone reproducer that boots a minimal Alpine VM and runs a single `wget` against a host-bound HTTP server. A successful run prints `pullrun-vm-outbound OK` to the guest’s serial console.
- **Grafana dashboard:** A 6-panel dashboard covering pull rate, workloads running, pull + start latency, exit code distribution, store size, and per-node runtime health.
- **Prometheus alerts:** Five alerts targeting common operational concerns (Table 4).

Table 4: Shipped Prometheus alerts.

Alert	Severity	Trigger
PullrunRuntimeDown	critical	Daemon not scraping for 2m
PullrunPullFailureRate	warning	>25% pulls failing for 5m
PullrunWorkloadCrashLoop	warning	Non-zero exits > 0.1/s for 10m
PullrunPullLatencyHigh	warning	p95 > 30s for 10m
PullrunStoreGrowingFast	info	Store growing > 1 GB/hour

13 Evaluation

We evaluated PULLRUN against Docker 24.0 on three platforms:

- **Mac mini M2 (2023):** Apple M2, 16 GiB RAM, macOS 14.5, 1 Gbps Ethernet.
- **Ubuntu 24.04 x86-64:** Intel Xeon E-2388G, 64 GiB RAM, KVM-enabled, 1 Gbps Ethernet.
- **Ubuntu 24.04 ARM64:** Ampere Altra, 32 GiB RAM, KVM-enabled (for Firecracker ARM64 testing).
- **Windows 11 x86-64 (WSL2):** Intel Core i9-13900K, 32 GiB RAM, WSL2 kernel 6.6, KVM-enabled, 1 Gbps Ethernet.

13.1 Pull Latency

We measured the time from `pullrun pull alpine:3.18` to completion versus `docker pull alpine:3.18`. Both systems were cold (empty store/daemon restart) and warm (image already present). Results are the mean of 10 trials.

Table 5: Pull latency comparison: PULLRUN vs. Docker.

Scenario	Docker	Pullrun	Speedup
Cold pull (alpine:3.18)	1,850 ms	968 ms	1.91×
Warm pull (alpine:3.18)	420 ms	12 ms	35×
Cold pull (ubuntu:22.04)	12.4 s	8.2 s	1.51×

PULLRUN’s warm pull advantage (12 ms vs 420 ms) reflects the DAG merge pattern: when all layer blobs are already present, the daemon writes a single `Manifest` node and exits. Docker must re-verify layer existence and update overlayfs metadata even for cached layers.

13.2 Memory Footprint

We measured daemon RSS at idle (no workloads running) using `ps -o rss`.

Table 6: Daemon RSS at idle.

Daemon	RSS
Docker Desktop 4.29 (macOS)	1,024 MiB
dockerd 24.0 (Linux)	82 MiB
PULLRUN (macOS, idle)	24.6 MiB
PULLRUN (Linux, idle)	18.2 MiB

PULLRUN’s minimal footprint follows from its storage architecture: no overlayfs mount table, no layer metadata cache (beyond the 256 MiB LRU), and no container runtime watcher background processes.

13.3 Boot Latency

We measured the time from workload creation to readiness (`status == running`) for each backend.

Table 7: Workload boot latency by backend (v0.3.2; Firecracker boot improved in v0.3.2 via cached ext4 image reuse).

Backend	Mean	Median	p99
Container (runc, Linux)	410 ms	395 ms	520 ms
Container (runc, WSL2/Win)	430 ms	415 ms	550 ms
Firecracker (KVM, Linux)	500 ms	480 ms	720 ms
Firecracker (KVM, WSL2/Win)	530 ms	510 ms	760 ms
Apple Virt (macOS M2)	160 ms	155 ms	210 ms

Apple Virtualization’s low boot time reflects the tight integration between the guest kernel and the hypervisor on Apple Silicon. Firecracker boot time improved significantly in v0.3.2 (~500 ms, down from ~1.24 s in v0.1.0) through ext4 image caching: the ext4 filesystem is built

once per image digest and reused on subsequent boots, eliminating the `mkfs.ext4` step from the critical path. The WSL2 path adds approximately 30 ms of socket overhead. Note that Apple Virt boot time includes rootfs materialization (hardlink creation, ~ 40 ms); the VM creation via `VZVirtualMachine::start()` completes in ~ 120 ms.

13.4 Storage Efficiency

We measured on-disk store size after pulling five common images:

Table 8: Storage efficiency comparison.

Image	Compressed	Docker overlayfs	Pullrun store
alpine:3.18	7.1 MiB	7.8 MiB	7.1 MiB
ubuntu:22.04	77.8 MiB	87.2 MiB	77.8 MiB
python:3.12	335 MiB	362 MiB	335 MiB
nginx:latest	187 MiB	203 MiB	187 MiB

PULLRUN’s store matches the compressed OCI size exactly because it stores raw layer blobs without extraction. Docker’s overlayfs extraction adds 10–15% overhead due to filesystem metadata and block alignment. When multiple images share base layers, Docker stores redundant copies due to separate overlayfs directories. PULLRUN’s content-addressed store deduplicates automatically.

13.5 P2P Sync Throughput

We measured the time to synchronize `ubuntu:22.04` (77.8 MiB compressed) from a single seed node to a consumer node over 1 Gbps Ethernet:

Table 9: P2P sync performance.

Method	Time
Pull from registry (cold)	8.2 s
P2P sync (Bloom + delta)	1.8 s
P2P sync (50% layers cached)	920 ms

The $4.6\times$ improvement over registry pull is due to local network bandwidth (~ 900 Mbps measured vs ~ 200 Mbps effective registry throughput) and the elimination of TLS overhead for intra-cluster transfers. With 50% of layers already cached, only the missing delta is transferred, reducing sync time to under 1 second.

13.6 Policy Engine Overhead

We measured the latency added by policy checks at `pull_image` time. The test image had a valid cosign signature and a CycloneDX SBOM with 150 components and 12 vulnerabilities.

The total policy gate adds <12 ms to the pull operation, negligible compared to network-bound pull times (hundreds of milliseconds to seconds). The engine runs in `tokio::task::spawn_blocking` because signature verification and SBOM parsing are CPU-bound.

13.7 Network Throughput

We measured cross-workload network throughput on the shared L2 segment:

Table 10: Policy engine latency (mean of 100 trials).

Check	Latency
Cosign signature verification (ed25519)	2.1 ms
SBOM parse (CycloneDX 1.5, 150 components)	8.3 ms
CVSS + license scan	1.4 ms
Total policy gate	11.8 ms

Table 11: Cross-workload network throughput (1 Gbps link).

Path	Throughput	Latency
Container → Container (same host)	985 Mbps	0.15 ms
Container → VM (same host)	972 Mbps	0.22 ms
VM → VM (same host)	968 Mbps	0.28 ms
Workload → Internet (via NAT)	942 Mbps	1.2 ms

The shared L2 bridge imposes minimal overhead (<2% throughput reduction) for intra-host communication. The userspace proxy adds ~1 ms latency for inbound connections due to the extra hop.

14 Practical Experience

14.1 Developer macOS and AI Agent Workflows

PULLRUN’s primary use case during development was replacing Docker Desktop for macOS developers. The 24.6 MiB daemon footprint (vs Docker Desktop’s ~1 GiB) translated to noticeably lower fan activity on MacBook Air systems. Apple Virt VM boot times of 160 ms made interactive terminal sessions feel instantaneous—comparable to native shell execution.

The VirtioFS rootfs sharing eliminated the “disk full” errors common with Docker Desktop’s Linux VM disk image. The rootfs directory is a host directory whose size is bounded only by available disk space, not a pre-allocated sparse image. Data volumes are shared natively via `VZVirtioFileSystemDeviceConfiguration` without FUSE proxy overhead.

The MCP server proved particularly useful for developers iterating on container configurations. Rather than memorising flag syntax, developers described workloads in natural language to Claude Code (“run the latest nginx image with port 8080 exposed and a volume at /tmp/data”), and the MCP server translated these into typed `run` tool calls. The `inspect` resource enabled agents to diagnose policy denials without leaving the editor.

The most significant operational friction is macOS code signing. The `com.apple.security.virtualization` entitlement requires the daemon binary to be signed with a hardened runtime entitlement. Apple’s linker strips code signatures during `cargo build`, requiring a post-build signing step:

```
codesign --force --sign - --entitlements virt.entitlements \
  --options runtime target/release/pullrun-runtime
```

We automated this via `make apple-sign-daemon` and documented it prominently, but it remains a point of friction for developers building from source.

14.2 Comparison with Docker and Podman

Table 12 summarises the key differentiators between PULLRUN, Docker CE, and Podman across the axes that matter most for PULLRUN’s target use cases.

Table 12: Feature comparison: PULLRUN 0.3.2 vs Docker CE 26 vs Podman 5.x.

Feature	Docker CE	Podman	Pullrun
Rootless by default	×	✓	✓
Daemonless	×	✓	✓
OCI-native storage	×	×	✓
Zero-copy warm pull	×	×	✓
Firecracker from OCI	×	×	✓
Apple Virt from OCI	×	×	✓
Windows container (WSL2)	✓	✓	✓
Windows Firecracker (WSL2)	×	×	✓
P2P image distribution	×	×	✓
Cosign + SBOM gating	×	partial	✓
Native MCP (in binary)	×	×	✓
Kubernetes CRI shim	via containerd	✓	✓
Compose support	✓	✓	✓
Ecosystem maturity	high	high	early

The × entry for Podman’s Cosign/SBOM column reflects that Podman supports cosign verification via `policy.json` image signing policies [12] but has no built-in SBOM scanning or CVSS-gating at the runtime level. The “native MCP” row distinguishes between PULLRUN’s compiled-in `pullrun mcp` subcommand (versioned with the daemon, zero external dependencies) and the community `podman-mcp-server` npm wrapper (separate install, separate versioning, external process).

14.3 CI/CD Integration

We deployed PULLRUN in a GitHub Actions self-hosted runner (Ubuntu x86-64, 4 vCPU, 16 GiB RAM) running a CI pipeline with 8 parallel container workloads (unit tests for each crate). Key observations:

- **Total pull time** for 8 parallel workloads was 3.1 s with DAG merge (all layers already cached) vs 22 s with Docker (overlayfs extraction).
- **No overlayfs errors**—we previously encountered “device or resource busy” errors on Docker due to stale overlayfs mounts after runner restarts. PULLRUN’s hardlink-based materialization has no such issue.
- **Memory stability**—daemon RSS remained at ~28 MiB under load (8 concurrent containers), compared to dockerd’s ~120 MiB + per-container overlayfs metadata overhead.

14.4 Kubernetes Deployment

We tested the CRI shim on a 3-node KIND cluster (Kubernetes v1.29). The RuntimeClass integration allowed mixing container and VM workloads in the same deployment:

- **Container workloads:** Pods with `runtimeClassName: pullrun-container` started in ~400 ms, comparable to containerd.

- **VM workloads:** Pods with `runtimeClassName: pullrun-vm` started in ~ 1.2 s as Firecracker microVMs, providing hardware isolation for untrusted code execution.
- **Image sharing:** Both backend types consumed the same OCI images from the same DAG store, eliminating the need for separate VM image pipelines.

The P2P sync layer reduced registry egress by $8\times$ during a rolling update of 10 pods: the first node pulled the new image from the registry, and the remaining 9 nodes delta-synced from peers.

14.5 Windows Developer Workflow

We tested PULLRUN 0.3.2 on Windows 11 (Intel i9-13900K, 32 GiB RAM) with WSL2 kernel 6.6. The auto-provisioning flow completed in 8 seconds on first run. Subsequent daemon starts (WSL2 already warm) took under 500 ms. Developers on the team using Windows noted that the absence of Docker Desktop (and its Hyper-V overhead) freed approximately 1.5 GiB of RAM and eliminated the “Starting Docker Desktop” latency that had previously added 20–30 seconds to their morning workflow.

The `-volume C:\Users\mo\code:\workspace` path translation worked without configuration: the DrvFs bind mount provided near-native I/O performance for source code directories. For file-intensive builds, we measured a 12% throughput reduction compared to native Linux due to the WSL2 DrvFs overhead, consistent with published WSL2 benchmarks.

Firecracker on WSL2 required no additional setup on Intel/AMD hosts: KVM was available out of the box in WSL2 6.6. The container and VM backends were immediately available after `pullrun.exe install`.

14.6 Policy Enforcement in Production

We configured the policy engine with `--require-signature --require-sbom --max-cvss 7.0` on a staging cluster. Over a 30-day period:

- 3 images were denied at pull time due to missing cosign signatures (development images that had not been through the CI signing pipeline).
- 1 image was denied due to a CVSS 8.1 vulnerability in a downstream dependency (detected via CycloneDX SBOM scan).
- 0 false positives: all denials were legitimate policy violations caught before any workload was created.

The policy engine’s observability integration proved valuable: `PolicyDenied` events were captured by the event bus and alerted via Prometheus, enabling the security team to respond to violations within minutes of occurrence.

15 Related Work

Container runtimes. `runc` [7] is the reference OCI runtime implementation, used by Docker, containerd, and Podman. `crun` [15] is a faster C implementation replaceable for `runc`. Neither supports VM backends. Kata Containers [16] runs each container in its own lightweight VM but requires a container-to-VM image conversion step and does not share a single storage engine between container and VM modes.

Podman [11] deserves dedicated comparison because it shares several of PULLRUN’s design goals—rootless by default, daemonless architecture, OCI-native—while diverging sharply in storage model and VM support. We examine the comparison systematically.

Shared ground. Both Podman and PULLRUN are rootless by default, require no privileged daemon, use runc as their container executor, and consume OCI images directly. Both are Apache-2.0 licensed and support Linux, macOS, and Windows. In terms of developer experience, both offer a Docker-compatible CLI surface.

Storage architecture. Podman delegates image storage to `containers/storage` [14], which extracts OCI layers into overlays or VFS lower directories—the same extraction-based model as Docker. This means Podman inherits the same $O(\text{image size})$ pull cost, the same overlays CVE exposure (CVE-2023-0386, CVE-2023-32629, CVE-2026-31431), and the same warm-pull overhead that PULLRUN’s DAG store eliminates. PULLRUN’s zero-copy mmap store and 12 ms warm pull latency have no equivalent in Podman.

VM support: the critical distinction. Podman offers `podman machine`, which provisions a **Linux infrastructure VM** (via QEMU on Linux, Apple Hypervisor on macOS, Hyper-V or WSL2 on Windows) to host the container runtime on non-Linux platforms [12]. This is categorically different from PULLRUN’s VM backend. The `podman machine` VM is a *host*—an invisible Linux layer that the operator never directly manages—not a *workload*. Running a workload as a microVM in Podman requires integrating a separate system such as Kata Containers, which introduces the image conversion step described above. PULLRUN runs the same OCI image as a Firecracker microVM *directly*, with no conversion, no nested runtime, and no separate image format. To our knowledge, no version of Podman offers Firecracker microVM workload execution from OCI images without an external adapter.

P2P distribution. Podman has no peer-to-peer image distribution mechanism. Clusters running Podman pull independently from registries, with no Bloom-filtered delta sync.

MCP integration. A community-maintained `podman-mcp-server` npm package [13] wraps the Podman socket via an external process. It is not part of the Podman binary, has no release guarantees aligned with Podman versions, and must be installed and configured separately. PULLRUN ships MCP as a first-class subcommand (`pullrun mcp`) compiled into the same binary as the runtime, with a versioned tool schema that evolves with the daemon’s gRPC API. No external process, no separate install, no version skew. To our knowledge, PULLRUN is the only container runtime that ships native MCP support as a core runtime primitive rather than a third-party wrapper.

Summary. Podman and PULLRUN are architecturally complementary rather than directly competitive for most of PULLRUN’s target workloads. A team running containers on a single Linux node with no VM isolation requirement and no P2P distribution need will find Podman’s ecosystem maturity (Podman Desktop, `podman generate kube`, Quadlets, 3M+ downloads as of early 2026) more immediately useful. A team requiring Firecracker isolation from OCI images, zero-copy storage, cluster-wide P2P sync, or native AI agent integration will find those capabilities absent from Podman.

Image distribution. Dragonfly [17] provides P2P image distribution for containers via a supernode architecture with pre-fetching and peer-to-peer block exchange. Unlike PULLRUN’s Bloom-filtered delta sync, Dragonfly uses a centralized supernode for metadata management. Kraken [18] (Uber) uses a BitTorrent-inspired protocol for image distribution but requires a separate cluster of tracker nodes. PULLRUN’s approach is fully decentralized: any node can seed any image without infrastructure. Spiegel [28] provides registryless image distribution for Kubernetes via a pull-through proxy on each node, but does not implement delta sync or Bloom-filtered block exchange.

Content-addressed storage. Git [19] popularized content-addressed object storage for source code. IPFS [20] generalizes this to arbitrary data with a distributed hash table. Within the container ecosystem, nydus [21] acceleration uses content-addressed chunked layers for lazy loading. PULLRUN’s store shares the same content-addressed philosophy but optimizes for the OCI layer model with rkyv-based zero-copy reads rather than chunk-level deduplication.

macOS virtualization. Lima [5] and Finch provide macOS Linux VMs via QEMU and

Apple Virtualization.framework. Unlike PULLRUN, they run containers *inside* the VM (nested) rather than sharing the image format between host and VM. Docker Desktop uses a hidden Linux VM via HyperKit. All three approaches incur the memory and CPU overhead of a full Linux VM even for container workloads. PULLRUN eliminates this by running container workloads natively (via runc on Linux or Apple Virt on macOS) with no intermediate VM for the container case.

Supply chain security. Sigstore/cosign [24] provides keyless signing and verification for container images. PULLRUN integrates cosign signature verification directly into the pull and run paths, rather than requiring a separate admission controller. The Open Policy Agent (OPA) [25] and Sigstore Policy Controller provide more expressive policy languages but require additional infrastructure. PULLRUN’s policy engine is intentionally simpler: it enforces cryptographic identity and SBOM properties locally without external dependencies.

Lazy loading. eStargz [26] and SOCI [27] enable lazy pulling of container image layers, fetching contents on demand rather than extracting everything upfront. PULLRUN takes the opposite approach: it pulls everything upfront but stores it without extraction, achieving fast warm pulls and zero runtime deserialization overhead. The two approaches are complementary: a future PULLRUN version could integrate lazy fetching on top of the DAG store.

16 Limitations and Future Work

PULLRUN v0 is a research prototype with several known limitations that point to future work:

Lazy loading. PULLRUN currently pulls entire layers before execution. For very large images (multi-GB ML models), lazy fetching of individual files on first access would reduce pull time further. This can be built on top of the existing DAG store by adding a fetch-on-miss handler to the materialization layer.

Multi-tenant networking. The shared L2 bridge model assumes all workloads are owned by the same operator. True multi-tenant isolation requires per-workload network namespaces, eBPF-based policies, and cross-node overlay networking. We plan to integrate Cilium’s eBPF datapath in v1.

Remote policy service. The v0 policy engine is fully local. v1 will support a sidecar that calls out to OPA, Cedar, or the Sigstore Policy Controller for richer rules (Rego, Cedar policies) while retaining the local gate for availability.

ARM64 Windows Firecracker. The Firecracker backend on Windows requires KVM, which WSL2 does not expose on ARM64 hosts as of Windows 11 24H2. ARM64 Windows users are limited to the container backend until Microsoft enables nested KVM in WSL2 on ARM.

Live migration. Firecracker supports live migration via its snapshot/restore mechanism. Integrating this into PULLRUN would enable workload migration across nodes without downtime, useful for cluster autoscaling and host maintenance.

Wasm backend. The `Backend::Wasm` variant is defined but not yet implemented. A `wasmtime`-based executor would enable OCI images containing Wasm modules to run directly, providing a fourth isolation level with sub-millisecond cold starts.

Registry write-back. The P2P sync protocol currently only distributes images that have been pulled from a registry. A push-to-registry feature would enable PULLRUN to function as a full registry mirror, writing locally-built images back to the upstream.

17 Conclusion

We presented PULLRUN, a content-addressed workload execution system that unifies containers, Firecracker microVMs, and Apple Silicon VMs under a single OCI-based storage and execution model. The zero-copy DAG store eliminates the extraction overhead inherent in existing container runtimes while providing $O(1)$ read access to any image layer. The unified executor abstraction

enables backend selection without image conversion. The P2P block sync layer provides scalable image distribution without registry bottlenecks. The policy engine adds defense-in-depth security gating via cosign signatures, SBOM analysis, and CVSS scoring. The Windows/WSL2 backend extends the same store, executors, and CLI to Windows 10/11, making PULLRUN the first container runtime to offer Firecracker microVMs on Windows without Docker Desktop. The native MCP server exposes the full runtime API to AI agents, enabling natural-language workload management without external scripting layers. The shared network model places all workloads on a single L2 segment with unified service discovery and port proxying.

PULLRUN demonstrates that the OCI image format can be the universal substrate for workload execution—not just for containers, but for all isolation levels. By treating content-addressed storage as a first-class primitive, we eliminate the format fragmentation, extraction overhead, and registry bottlenecks that have accumulated in the container stack over the past decade.

PULLRUN 0.3.2 is open-source and available at <https://github.com/pullrun/pullrun>. The source code archived at the version corresponding to this technical report is permanently available via Zenodo [29] at <https://doi.org/10.5281/zenodo.20679669>.

Acknowledgments

The author thanks the Rust and Go open-source communities for the ecosystem of libraries that made PULLRUN possible, including rkyv, tokio, tonic, clap, memmap2, dashmap, and cobra. The Firecracker team at Amazon Web Services provided critical documentation and design guidance for the KVM VM backend. Apple’s Virtualization.framework documentation enabled the macOS backend. The author is grateful to early adopters who provided feedback on the developer experience and Kubernetes integration.

References

- [1] Docker Inc., “Docker Hub Statistics,” 2024. <https://www.docker.com/blog/docker-hub-statistics/>
- [2] Open Container Initiative, “OCI Image Format Specification v1.1.0,” 2024. <https://github.com/opencontainers/image-spec>
- [3] K. Sato et al., “Deduplicating Container Image Downloads with eStargz,” in *ATC 2021*, USENIX.
- [4] A. Agache et al., “Firecracker: Lightweight Virtualization for Serverless Applications,” in *NSDI 2020*, USENIX.
- [5] Lima contributors, “Lima: Linux Virtual Machines,” 2024. <https://github.com/lima-vm/lima>
- [6] D. Koloski, “rkyv: Zero-Copy Deserialization Framework,” 2024. <https://github.com/rkyv/rkyv>
- [7] Open Container Initiative, “runc: CLI tool for spawning and running containers,” 2024. <https://github.com/opencontainers/runc>
- [8] S. Bruno, “passt: Plug A Simple Socket Transport,” 2023. <https://passt.top/>
- [9] Apple Inc., “Virtualization.framework Documentation,” 2024. <https://developer.apple.com/documentation/virtualization>

- [10] A. Das, I. Gupta, and A. Motivala, “SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol,” in *DSN 2002*, IEEE.
- [11] Red Hat Inc., “Podman: A tool for managing OCI containers and pods,” 2024. <https://podman.io/>
- [12] Red Hat Inc., “Podman Machine Documentation,” 2024. <https://docs.podman.io/en/latest/markdown/podman-machine.1.html>
- [13] Community contributors, “podman-mcp-server: MCP server for Podman and Docker,” 2025. <https://github.com/containers/podman-mcp-server>
- [14] Containers project, “containers/storage: Go library for local image and container storage,” 2024. <https://github.com/containers/storage>
- [15] G. Scano, “crun: A fast and lightweight OCI runtime,” 2024. <https://github.com/containers/crun>
- [16] Kata Containers contributors, “Kata Containers,” 2024. <https://katacontainers.io/>
- [17] Dragonfly contributors, “Dragonfly: P2P-based image and file distribution system,” 2024. <https://d7y.io/>
- [18] Uber Technologies, “Kraken: P2P Docker Registry,” 2019. <https://eng.uber.com/kraken/>
- [19] L. Torvalds, “Git: A content-addressable filesystem,” 2005. <https://git-scm.com/>
- [20] J. Benet, “IPFS: Content Addressed, Versioned, P2P File System,” 2014. arXiv:1407.3561.
- [21] Alibaba Group, “nydus: Dragonfly Container Image Service,” 2024. <https://github.com/dragonflyoss/nydus>
- [22] Docker Inc., “Docker: Accelerated Container Application Development,” 2024. <https://www.docker.com/>
- [23] Containerd contributors, “containerd: An open and reliable container runtime,” 2024. <https://containerd.io/>
- [24] Sigstore contributors, “Sigstore: A new standard for signing, verifying and protecting software,” 2024. <https://www.sigstore.dev/>
- [25] Open Policy Agent contributors, “Open Policy Agent,” 2024. <https://www.openpolicyagent.org/>
- [26] K. Sato and K. Matsumoto, “Accelerating Container Image Distribution with eStargz,” 2020. <https://github.com/containerd/stargz-snapshotter>
- [27] AWS SOCI contributors, “Seekable OCI (SOCI),” 2023. <https://github.com/awslabs/soci-snapshotter>
- [28] X. Granade et al., “Spegel: Stateless cluster local OCI registry mirror,” 2023. <https://github.com/spegel-org/spegel>
- [29] M. Boukaba, “Pullrun v0.3.2: A Content-Addressed Workload Execution System,” Zenodo, June 2026. DOI: [10.5281/zenodo.20679669](https://doi.org/10.5281/zenodo.20679669)
- [30] Microsoft Corporation, “Windows Subsystem for Linux 2,” 2024. <https://learn.microsoft.com/en-us/windows/wsl/>

- [31] Anthropic, “Model Context Protocol Specification,” 2024. [https://
modelcontextprotocol.io/](https://modelcontextprotocol.io/)
- [32] MITRE Corporation, “CVE List,” 2024. <https://cve.mitre.org/>